CONDITIONALS, LOOPS

BROWSER API

```
alert('message'); // shows a message

prompt('input'); // shows a message asking the user to input t

confirm('question'); // shows a message and waits for the user

console.log('message'); // prints a message to console

console.error('error message'); // prints an error message to
```

JAVASCRIPT COMPARISON OPERATORS

Comparison operators compare two values and return a boolean value (true or false)

```
const a = 3;
const b = 2;
console.log(a > b); // true
```

a > b is called a boolean expression since evaluating it results in a boolean value.

COMMONLY USED COMPARISON OPERATORS

Operator	Meaning	Example	Boolean Conversion
==	Equal to	3 == 5 // false	true
!=	Not equal to	3 != 4 // true	false
===	Strictly equal to	3 === "3" // false	true
!==	Strictly not equal to	3 !== "3" // true	true
>	Greater than	4 > 4 // false	true
<	Less than	3 < 3 // false	true
>=	Greater than or equal to	4 >= 4 // true	false
<=	Less than or equal to	3 <= 3 // true	false

JAVASCRIPT EQUAL TO OPERATOR

The equal to operator == evaluates to

- true if the values of the operands are equal.
- false if the values of the operands are not equal.

```
// same value, same type
console.log(5 == 5); // true

// same value, different type
console.log(2 == '2'); // true

// different values, same type
console.log('hello' == 'Hello'); // false
```

JAVASCRIPT NOT EQUAL TO OPERATOR

The not equal to operator != evaluates to

- true if the values of the operands aren't equal.
- false if the values of the operands are equal.

```
// same value, same type
console.log(2 != 2); // false

// same value, different type
console.log(2 != '2'); // false

// different value, same type
console.log(2 != 3); // true
```

STRICT EQUAL TO OPERATOR

The strict equal to operator === evaluates to

- true if both the values and the types of the operands are the same.
- false if either the values or the types of the operands are not the same.

```
// same value, same type
console.log(2 === 2); // true

// same value, different type
console.log(2 === '2'); // false
```

DIFFERENCE BETWEEN == AND === OPERATORS

The == (equality) operator only checks the values of the operands and not their types. For example,

However, the === (strict equality) operator checks both the values and types of the operands. For example,

```
// only checks the values
console.log(2 == '2'); // true

// checks both the values and the types
console.log(2 === '2'); // false
```

STRICT NOT EQUAL TO OPERATOR

The strict not equal to operator !== evaluates to

- true if either the values or the types of the operands are not the same.
- false if both the values and the types of the operands are the same.

```
// same value, same type
console.log(2 !== 2); // false

// same value, different type
console.log(2 !== '2'); // true

// different value, same type
console.log('Hello' !== 'World'); // true
```

GREATER THAN OPERATOR

The greater than operator > returns

- true if the value on the left is greater than the value on the right.
- false if the value on the left isn't greater than the value on the right.

```
// left operand is greater
console.log(3 > 2); // true

// both operands are equal
console.log(4 > 4); // false

// left operand is smaller
console.log(2 > 5); // false
```

GREATER THAN OR EQUAL TO OPERATOR

The greater than or equal to operator >= returns

- true if the value on the left is greater than or equal to the value on the right.
- false if the value on the left is less than the value on the right.

```
// left operand is greater
console.log(3 >= 2); // true

// both operands are equal
console.log(4 >= 4); // true

// left operand is smaller
console.log(2 >= 5); // false
```

LESS THAN OPERATOR

The less than operator < returns

- true if the value on the left is less than the value on the right.
- false if the value on the left isn't less than the value on the right.

```
// left operand is smaller
console.log(2 < 5); // true

// both operands are equal
console.log(4 < 4); // false

// left operand is greater
console.log(3 < 2); // false
```

JAVASCRIPT LOGICAL OPERATORS

Logical operators return a boolean value by evaluating boolean expressions

Syntax	Description	Boolean Conversion
expression1 && expression2	true only if both expression1 and expression2 are true	true
expression1 || expression2	true if either expression1 or expression2 is true	false
!expression	false if expression is true and vice versa	true

OPERATOR PRECEDENCE

Which operator gets called first? Operators are called in order of precedence (higher precedence wins)

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Operator

OPERATOR ASSOCIATIVITY

When operators have the same precedence, what order operators get called in: left-to-right or right-to-left

```
3 + 4 * 5; // 23, first *, then +
```

OPERATOR PRECEDENCE AND ASSOCIATIVITY

Question

```
console.log(3 + 10 * 2);
console.log((3 + 10) * 2);
```

OPERATOR PRECEDENCE AND ASSOCIATIVITY

Answer

```
console.log(3 + 10 * 2); // 23
console.log((3 + 10) * 2); // 26, because the parentheses chan
```

OPERATOR PRECEDENCE AND ASSOCIATIVITY

Question

```
const a = 1;
const b = 2;
typeof a + b;
```

OPERATOR PRECEDENCE AND ASSOCIATIVITY

Answer

```
const a = 1;
const b = 2;
typeof a + b; // Equivalent to (typeof a) + b; result is "numb
```

CONDITIONS, IF-ELSE, TERNARY

JAVASCRIPT IF STATEMENT

Use if keyword to execute code based on some specific condition. The if keyword checks the condition inside the parentheses ().

- If the condition is evaluated to true, the code inside { } is executed.
- If the condition is evaluated to false, the code inside { } is skipped.

```
if (condition) {
  // block of code
}
```

JAVASCRIPT IF STATEMENT EXAMPLE

```
// Program to check if the number is positive

const number = prompt('Enter a number: ');

// check if number is greater than 0
if (number > 0) {
  // the body of the if statement
  console.log('positive number');
}

console.log('nice number');
```

JAVASCRIPT ELSE STATEMENT

Use else keyword to execute code when the condition specified in the preceding if statement evaluates to false.

- If condition is true, the code inside if is executed. And, the code inside else is skipped.
- If condition is false, the code inside if is skipped. Instead, the code inside else is executed.

```
if (condition) {
  // block of code
  // execute this if condition is true
} else {
  // block of code
  // execute this if condition is false
}
```

JAVASCRIPT IF..ELSE STATEMENT EXAMPLE

```
let age = 17;

// if age is 18 or above, you are an adult
// otherwise, you are a minor

if (age >= 18) {
  console.log('You are an adult');
} else {
  console.log('You are a minor');
}

// Output: You are a minor
```

JAVASCRIPT ELSE..IF STATEMENT

Use the else if keyword to check for multiple conditions.

```
// check for first condition
if (condition1) {
  // if body
}

// check for second condition
else if (condition2) {
  // else if body
}

// if no condition matches
else {
  // else body
}
```

JAVASCRIPT ELSE IF STATEMENT EXAMPLE

```
let rating = 4;

// rating of 2 or below is bad
// rating of 4 or above is good
// else, the rating is average

if (rating <= 2) {
  console.log('Bad rating');
} else if (rating >= 4) {
  console.log('Good rating!');
} else {
  console.log('Average rating');
}

// Output: Good rating!
```

JAVASCRIPT NESTED IF...ELSE STATEMENT

When we use an if...else statement inside another if...else statement, we create a nested if...else statement

```
let marks = 60;

// outer if...else statement
// student passed if marks 40 or above
// otherwise, student failed

if (marks >= 40) {
  // inner if...else statement
  // Distinction if marks is 80 or above

  if (marks >= 80) {
    console.log('Distinction');
  } else {
    console.log('Passed');
  }
} else {
  console.log('Failed');
}
```

JAVASCRIPT TERNARY OPERATOR ?

We can use the ternary operator ?: instead of an if...else statement if the operation we're performing is very simple

```
let grade = 40;
let result;

if (grade >= 50) {
  result = 'pass';
} else {
  result = 'fail';
}

console.log(result);
```

```
let grade = 40;

let result = grade >= 50 ? 'pass' : 'fail';

console.log(result);
```

JAVASCRIPT SWITCH-CASE OPERATOR

We can replace our if…else statement with the switch statement when we deal with a large number of conditions.

```
let grade = 'C';

// using if else for many conditions
// first condition
if (grade === 'A') {
  console.log('Excellent!');
}
// second condition
else if (grade === 'B') {
  console.log('Good!');
}
// third condition
else if (grade === 'C') {
  console.log('Average');
}
// fourth condition
else if (grade === 'D') {
  console.log('Bad');
}
```

JAVASCRIPT SWITCH-CASE OPERATOR

We can replace our if...else statement with the switch statement when we deal with a large number of conditions.

```
let grade = 'C';

// using switch...case
switch (grade) {
  // first condition
  case 'A':
    console.log('Excellent!');
    break;
  // second condition
  case 'B':
    console.log('Good!');
    break;
  // third condition
  case 'C':
    console.log('Average');
    break;
  // fourth condition
  case 'D':
    console.log('Bad');
```

LOOPS

JAVASCRIPT FOR LOOP

the for loop is used for iterating over a block of code a certain number of times

```
for (let i = 0; i < 3; i++) {
  console.log('Hello, world!');
}

// Output:
// Hello, world!
// Hello, world!
// Hello, world!
```

JAVASCRIPT FOR LOOP

The syntax of the for loop is:

```
for (initialExpression; condition; updateExpression) {
  // for loop body
}
```

- initialExpression - Initializes a counter variable.
- condition - The condition to be evaluated. If true, the body of the for loop is executed.
- updateExpression - Updates the value of initialExpression.

Once an iteration of the loop is completed, the condition is evaluated again. The process continues until the condition is false.

JAVASCRIPT FOR LOOP EXAMPLE 1

Print numbers from 1 to 5 using a for loop.

```
for (let i = 1; i < 6; i++) {
  console.log(i);
}
```

Iteration	Variable	Condition: i < 6	Action
1st	i = 1	true	1 is printed. i is increased to 2.
2nd	i = 2	true	2 is printed. i is increased to 3.
3rd	i = 3	true	3 is printed. i is increased to 4.
4th	i = 4	true	4 is printed. i is increased to 5.
5th	i = 5	true	5 is printed. i is increased to 6.
6th	i = 6	false	The loop is terminated.

JAVASCRIPT FOR LOOP EXAMPLE 2

Print sum of n Natural Numbers

```
// program to display the sum of natural numbers

let sum = 0;
const n = 100;

// loop from i = 1 to i = n
// in each iteration, i is increased by 1
for (let i = 1; i <= n; i++) {
  sum += i; // sum = sum + i
}

console.log(`sum: ${sum}`);

// Output: sum: 5050
```

Initially, the value of sum is 0, while n has a constant value of 100.

Then, we iterate a for loop from i = 1 to n. In each iteration,

- i is added to sum.
- Then, the value of i is increased by 1.
- When i becomes 101, the test condition becomes false and sum will be equal to 0 + 1 + 2 + ... + 100.

JAVASCRIPT NESTED FOR LOOPS

A for loop can also have another for loop inside it. For each cycle of the outer loop, the inner loop completes its entire sequence of iterations. For example,

```
// outer loop
for (let i = 0; i < 3; i++) {
  // inner loop
  for (let j = 0; j < 2; j++) {
    console.log(`i = ${i}, j = ${j}`);
  }
}
```

Output

```
i = 0, j = 0
i = 0, j = 1
i = 1, j = 0
i = 1, j = 1
i = 2, j = 0
i = 2, j = 1
```

- Outer Loop - Runs from i = 0 to 2.
- Inner Loop - Runs from j = 0 to 1.

In each iteration of the outer loop, the inner loop runs from j = 0 to 1.

JAVASCRIPT INFINITE FOR LOOP

We can create an infinite for loop by setting a condition that always evaluates to true

```
for (let i = 0; true; i++) {
  console.log('This loop will run forever!');
}
```

Output

```
This loop will run forever!
This loop will run forever!
This loop will run forever!
…
```

In this example, the condition in the for loop is explicitly set to true.

Since this condition never changes and always evaluates to true, the loop will continue indefinitely (until the memory is full).

JAVASCRIPT OMITTING PARTS OF THE FOR LOOP

We can omit any part of the for loop declaration and include it in a different part of the code.

```
// initialization outside the loop
let i = 0;

// omit initialization and update statements
for (; i < 3; ) {
  console.log(`i is ${i}`);

  // increment inside the loop body
  i++;
}
```

Here, we have initialized i before the loop, which iterates as long as i is less than 3.

Notice that i is incremented within the loop body, which allows us to skip the update statement inside the parentheses () of the for loop.

In other words, for (; i < 3;) indicates omitted initialization and update expression, focusing only on the condition.

JAVASCRIPT WHILE LOOP

The while loop repeatedly executes a block of code as long as a specified condition is true.

```
while (condition) {
  // body of loop
}
```

1. The while loop first evaluates the condition inside ( ).
2. If the condition evaluates to true, the code inside { } is executed.
3. Then, the condition is evaluated again.
4. This process continues as long as the condition evaluates to true.
5. If the condition evaluates to false, the loop stops.

JAVASCRIPT WHILE LOOP EXAMPLE

```
// initialize variable i
let i = 1;

// loop runs until i is less than 4
while (i < 4) {
  console.log(i);
  i += 1;
}
```

```
1
2
3
```

Variable	Condition: i < 4	Action	Action
i = 1	true	1 is printed. i is increased to 2.	1 is printed. i is increased to 2.
i = 2	true	2 is printed. i is increased to 3.	2 is printed. i is increased to 3.
i = 3	true	3 is printed. i is increased to 4.	3 is printed. i is increased to 4.
i = 4	false	The loop is terminated.	4 is printed. i is increased to 5.
5th	i = 5	true	5 is printed. i is increased to 6.
6th	i = 6	false	The loop is terminated.

JAVASCRIPT INFINITE WHILE LOOP

An infinite while loop is a condition where the loop runs infinitely, as its condition is always true

```
let i = 1;

// always true condition
while (i < 5) {
  console.log(i);
}
```

the condition 1 > 1 is always true, which causes the loop body to run forever.

JAVASCRIPT BREAK STATEMENT

The break statement terminates the loop immediately when it's encountered.

```
// Program to print the value of i

for (let i = 1; i <= 5; i++) {
  // break condition
  if (i == 3) {
    break;
  }

  console.log(i);
}
```

Output

```
1
2
```

JAVASCRIPT BREAK STATEMENT

The break statement terminates the loop immediately when it's encountered.

```
let sum = 0;
let num = 5;

// infinite loop
while (true) {
  if (num < 0) {
    // terminate the loop if num is negative
    break;
  } else {
    // otherwise, add num to sum
    sum += num;
  }

  num--; // decrease num, equals to num = num - 1;
}

// print the sum
console.log(`Sum: ${sum}`); // 15, because 5 + 4 + 3 + 2 + 1 + 0
```

Output

```
Sum: 15
```

JAVASCRIPT BREAK WITH NESTED LOOPS

When break is used inside two nested loops, it terminates the inner loop

```
// nested for loops

// outer loop
for (let i = 1; i <= 3; i++) {
  // inner loop
  for (let j = 1; j <= 3; j++) {
    if (i == 2) {
      break;
    }
    console.log(`i = ${i}, j = ${j}`);
  }
}
```

Output

```
i = 1, j = 1
i = 1, j = 2
i = 1, j = 3
i = 3, j = 1
i = 3, j = 2
i = 3, j = 3
```

JAVASCRIPT CONTINUE STATEMENT

The continue statement skips the current iteration of the loop and proceeds to the next iteration.

```
for (let i = 1; i <= 10; ++i) {
  // skip iteration if value of
  // i is between 4 and 9
  if (i > 4 && i < 9) {
    continue;
  }
  console.log(i);
}
```

Output

```
1
2
3
4
9
10
```

JAVASCRIPT CONTINUE STATEMENT

The continue statement skips the current iteration of the loop and proceeds to the next iteration.

```
let num = 1;

while (num <= 10) {
  // skip iteration if num is even
  if (num % 2 === 0) {
    ++num;
    continue;
  }

  console.log(num);
}
```

Output

```
1
3
5
7
9
```

USED MATERIALS

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Refere

https://www.programiz.com/javascript/for-loop

https://www.programiz.com/javascript/while-loop

https://www.programiz.com/javascript/break-statement

https://www.programiz.com/javascript/continue-statement

https://www.w3schools.com/js/js_if_else.asp

https://www.w3schools.com/js/js_loop_for.asp

https://www.w3schools.com/js/js_loop_while.asp